

**INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE
MONTERREY**



**Tecnológico
de Monterrey**

Analysis and Design of Advanced Algorithms

Integrated Activity 1

DIEGO RODRIGUEZ SANCHEZ (A01742085)

Introduction

The problem situation describes a scenario where information sent between two devices travels as a stream of bits, carrying a header, the actual data, and a tail. A malicious actor can intercept this stream, alter parts of it, and forward it to the receiver with its own code inside of it, which can later be used to take control over the device which receives the information.

The following activity asks the team to build a program that supports three tasks related to that scenario: detecting whether known malicious code sequences appear inside a transmission and where, identifying a distinctive pattern (a palindrome) that could mark the start of the real data inside the transmission, and comparing two transmissions to find the longest sequence they share, which could point to a common piece of inserted code. The three algorithms discussed below are the ones we used to address those three tasks.

1. Pattern Search

Problem Description

The first part of the activity asks the program to check if the malicious code parts (mcodes) appear as an exact sequence of characters inside each transmission file, and, when they report the position where the match starts. This has to be done six times because we are asked to check the 3 mcodes into the 2 different transmissions, we should find the most efficient way to find those patterns in the transmissions even though if they are long.

Algorithm

If we try to compare the pattern into the transmissions comparing every position one by one, we are wasting time, because every time the matches fail, we move one character forward and start the comparison again. To solve this problem, we used KMP algorithm, the benefits of using it are that it avoids reusing the mismatch problems. If a part of the pattern beginning also reappears at the end suffix prefix, the pattern can slide forward directly skipping some comparisons.

In order to let the algorithm know how far it should go in order to skip those extra comparisons we needed to build an LPS (longest prefix suffix) array. For each position in the pattern, the length of the longest prefix that is also a suffix ending there. Because it depends only on the pattern it is built once before the text is touched. The search then uses two pointers, one on the text and one on the pattern. When characters match, both advance, but when they don't, the text pointer never moves backward. The LPS array moves the pattern pointer to the correct position instead, and the comparison continues from there. When the pattern pointer reaches the end of the pattern, a match is found and its starting position is reported.

Time and Space complexity

Building the LPS array takes $O(m)$ time: the inner loop can reassign the length variable, but it only decreases as many times as the index advances, so the total work is bounded by $2m$.

Searching the text takes $O(n)$ time by the same argument: the text pointer never moves backward, and the pattern pointer falls back only as many times as it previously advanced.

Total time complexity: $O(n + m)$, well below the $O(n \text{ times } m)$ worst case of the naive approach.

Space complexity: $O(m)$, for the LPS array, the only extra memory used.

Since the transmission files (n) are large and the malicious code files (m) are small, linear time in n with memory proportional to m keeps the program fast as transmissions grow.

2. Longest Palindromic Substring

Problem Description

The second part asks the program to find the longest palindrome (a sequence that reads the same forwards and backwards) inside each transmission file, and to report where it starts and ends. The comparison is done at the character level, using the actual characters, not their binary representation.

Algorithm

The first idea might be to expand around every position, growing outward while the characters match as we saw in class. That works, but it's $O(n^2)$, since nothing gets reused between positions, which isn't good enough for large files.

Manacher's algorithm solves this in $O(n)$ by taking advantage of the fact that palindromes are symmetric: once we know one, we already know something about the positions mirrored around its center, so we don't have to compare from scratch every time.

First, we transform the string by inserting a separator between every character and at both ends, so every palindrome ends up with odd length. This way we avoid handling even and odd lengths as separate cases. Then, as we move through the string, we keep track of the palindrome that reaches furthest to the right so far. For each position, if it falls inside that known palindrome, its mirrored position already gives us a length we can reuse — unless that length would stretch past what we actually know, in which case we only trust it up to that point. If the position falls outside the known palindrome, we start from zero instead. Either way, we then try to expand a bit further by comparing characters directly, and we update our reference palindrome if this one reaches further right. Throughout, we keep track of the best length found and where it occurred.

Finally we need to map the result back to the original string and report it using the 1-based numbering the activity asks for

.

Time and Space complexity

Building the transformed string takes $O(n)$ time, since it's a single pass over the original string.

The main loop looks like it could be $O(n^2)$ because of the while loop nested inside the for loop, but the inner loop doesn't run freely: every successful expansion pushes right one position further, and right never moves backward, so across the whole execution it can only succeed n times total, not n times per position. This makes the whole main loop $O(n)$.

Total time complexity: $O(n)$, a clear improvement over the $O(n^2)$ expand-around-center approach, and what makes it practical for large transmission files.

Space complexity: $O(n)$, for the transformed string and the array p , both proportional to the input size.

3. Longest Common Substring

Problem Description

The last part asks us to develop a program that aims to measure how similar the two transmission files are, by finding the longest sequence of characters that appears, contiguously, in both transmission files, and reporting where it starts and ends in the first file. This is a substring problem, not a subsequence problem: the matching characters must be consecutive, with no gaps allowed.

Algorithm

We solved this with (DP) dynamic programming, building a table where each cell answers different questions:

- If the match ends at the same position in both files
- How long is the match

Filling each cell follows two rules:

- If the characters match
- If the value grows by one from any match that exists, one position earlier

If any of these rules are broken a reset happens. That reset is what separates this from longest common subsequence, where a mismatch would carry the previous value forward instead of discarding it, since subsequences allow gaps and substrings don't.

Each of the cells only depends on the row right above it, so the full table is never needed, just by having the current row and the previous one we can swap after each pass. While filling each row we are tracking the largest value and where it occurred which give us the ending position of the first file, which is followed by the starting position. If there is no match found, we can now report both positions as zero. instead of computing a meaningless range.

Time and Space complexity

Filling the table takes $O(n \text{ times } m)$ time: the outer loop runs n times, the inner one m times, and each cell does constant work, one comparison, one assignment, one check against the running maximum.

Space complexity: $O(m)$, since only two arrays of size $m+1$ are kept, the current row and the previous one, instead of the full table.

This matters for the project because a full n by m table would need one integer per pair of positions, which grows fast for large transmission files; keeping only two rows keeps memory linear instead of quadratic.

Conclusion

Working on the findPattern algorithm was a little bit of a hard job, it was not that hard because we already saw it in the classroom and, I investigated we the mythic Abdul Bahri professor on YouTube. I think the hardest part to me was developing the logic behind the LPS construction. It was hard because I knew that we needed to store those prefixes, but I didn't know how the correct way is in which we needed to store those.

When the other algorithms were developed, I realized the importance of applying optimization while developing algorithms, and how important they are in several daily activities. It is incredible for me how by reusing certain data your algorithm is comparing lowers the time or space complexity. Also, I reaffirmed a phrase that I have heard, "There is no perfect algorithm, there are perfect situations for each algorithm." It is important to realize that you need to choose carefully which algorithm to use to solve certain problems and how to apply it.

Also something that resulted very important to me while working on this activity was to increase my abilities to understand the logic behind algorithms.